

Hardware-Based Identity Verification for Embedded Systems

Spring 2026 ECE 56800 Project Final Report

Clock Profilers

Michael Antabian, Husam Chekfa, (Lead) Dalen Ricks

Abstract

This project implements a hardware-based device fingerprinting system for IoT networks using clock skew as a physical identifier. Traditional cryptographic authentication imposes computational overhead on resource constrained embedded systems. Our approach exploits the fact that every device's quartz crystal oscillator has a unique manufacturing variation. This variation causes its clock to drift from true time at a slightly different rate. The variation is known as clock skew and it can be measured remotely by observing the TCP timestamp values carried in network packets. We implement a two-component system: an ESP32 microcontroller firmware that connects to a Wi-Fi network and transmits TCP packets with timestamps enabled via the lwIP stack, and a Python-based fingerprinter running on a laptop that captures these packets using Scapy, extracts the TCP timestamp values, and estimates the clock skew using a linear programming-based algorithm derived from Kohno et al. The system successfully captures and processes TCP timestamp pairs at a rate of one packet per second. Results from four ESP32 devices show that each device exhibits a measurably distinct clock skew, -1.63ppm, 14.94ppm, 19.46ppm, 7.74ppm, validating that clock skew can serve as a unique hardware fingerprint for device identification without any cryptographic overhead.

1 Introduction

As the world increasingly relies on wirelessly connected Internet of Things (IoT) devices, network security becomes increasingly important with it. IoT networks are susceptible to cyber attacks, of which one is called Sybil attacks. These cyber attacks involve a malicious actor that fabricates multiple identities to subvert network trust mechanisms [1]. The standard defense mechanism against such attacks usually involves cryptographic authentication schemes that verify identity and data integrity by requiring proof of a cryptographic key, rather than just a password. While these measures provide strong security guarantees, they require computational resources that might not be available on resource constrained embedded systems. For embedded systems with limited RAM, CPU cores, or battery capacity, the overhead of cryptographic authentication can be critical.

This project proposes and implements an alternative approach to network security. By using device identification that relies on a physical property of the hardware, fabricated identities won't be able to subvert network trust mechanisms. This hardware property is the clock skew, which is unique to every physical device. All digital devices contain a quartz crystal oscillator that drive their internal clock. Due to microscopic variations in the manufacturing process, no two crystals oscillate at exactly the same frequency. This means every device's clock drifts from true time at a slightly different rate, and this drift rate is unique to each device and remains stable over time [2].

Kohno et al. demonstrated that clock skew can be measured remotely by analyzing the TCP timestamp option values embedded in network packets [2]. Since each outgoing TCP packet carries a timestamp generated by the device's internal clock, an observer can track the drift of that clock over time by comparing the reported timestamps against the observed packet arrival times. The resulting skew estimate serves as a hardware fingerprint that cannot be easily spoofed. It's fingerprint is rooted in the physical properties of the oscillator rather than in software-managed credentials.

Our project implements this fingerprinting technique on the ESP32 platform as a way to validate the approach in an embedded systems context. Our system is designed to serve as a low power and low-cost network monitor that is capable of detecting devices across different networks or locations using their clock skew. It can be applied in network security measures by detecting identity spoofing and flagging devices whose measured clock skew does not match the profile associated with their reported MAC address.

2 Background

Remote device fingerprinting via clock skew was introduced by Kohno et al. [2], who showed that the TCP timestamps option (RFC 1323) provides a reliable mechanism for estimating a device's clock skew. Their technique collects pairs of values from observed TCP packets: the local arrival time recorded by the measurer, and the TCP timestamp value reported by the device. By fitting an upper-bounding line to the observed offset between the device's clock and real time over a series of packets, one can estimate the slope of that line, which corresponds to the clock skew. Kohno et al. demonstrated that these estimates are stable across many different conditions such as reboots, independent networks, and different technologies. They were distinct enough across different physical machines to serve as reliable fingerprints.

Huang et al. extended this concept to wireless sensor networks (WSN). Wireless Sensor Network devices lack TCP/IP stacks and instead use the Flooding Time Synchronization Protocol (FTSP) as a measurement tool [1]. Their work confirmed that every sensor node has a unique and stable clock skew, and they proposed a lightweight identification algorithm suitable for in-network deployment. They introduced a line continuity check for cases where two nodes have similar skew averages. This check tracks the variation between consecutive skew measurements to detect when a node's reported identity changes suddenly, which could signal a potential Sybil attack.

Existing cryptographic security measures for node authentication provide strong guarantees, but at a certain computational cost. Public-key infrastructure, pairwise key predistribution, and similar schemes all require nontrivial computation and memory, which may exceed the capabilities of the most constrained embedded

devices [1]. Clock skew-based fingerprinting requires no modification to the fingerprintee and no certificate management, making it appropriate to resource-constrained IoT environments.

3 Problem Statement

This project addresses the problem of uniquely identifying IoT devices on a shared Wi-Fi network without requiring cryptographic overhead on the monitored devices. Specifically, we aim to measure the clock skew of each device by passively observing its TCP packets and saving their skews to later on identify them.

The core challenge is extracting a stable and accurate skew estimate from a stream of network packets that are subject to jitter and variable transmission delays. A secondary challenge is confirming that the skew estimates for different physical devices are sufficiently distinct to be used as unique identifiers in practice. Once a device has been fingerprinted, its skew profile persists across network changes and reboots, since the underlying oscillator frequency does not change.

4 Proposed Approach

Our system consists of two main components: the *fingerprintee*, an ESP32 microcontroller that generates network traffic with TCP timestamps, and the *fingerprinter*, a Python-based measurement system running on a host device that captures and analyzes that traffic.

4.1 System Overview

The fingerprintee connects to a local Wi-Fi access point and establishes a persistent TCP connection to a measurement server running on the fingerprinter device. Once connected, it transmits periodic packets at a fixed interval. Each outgoing TCP packet carries a timestamp in the TCP options field, which gets generated by the device's internal clock from the lwIP TCP timestamps option. The fingerprinter runs two concurrent processes: a TCP server that accepts the ESP32's connection and keeps it alive, and a packet sniffer that captures the packets at the network level and records the relevant timing information. The recorded timing information is then used by a clock skew calculation program that will provide the device's unique skew.

4.2 ESP32 Firmware (Fingerprintee)

The fingerprintee firmware is written in C using the ESP-IDF framework, which provides FreeRTOS as the real-time operating system and lwIP as the TCP/IP network stack. The firmware performs three main tasks at startup: initializing NVS flash storage, connecting to the configured Wi-Fi access point, and establishing a TCP connection to the measurement server.

The ESP-IDF framework comes equipped with a graphical text-based user interface called menuconfig. It allows configuring certain parameters and project settings before building, without having to edit the code. Our firmware added custom parameters to the esp-idf menuconfig using the Kconfig.projbuild file. These parameters exposed configuration options for SSID, password, server IP address, and server port. The fingerprintee Wi-Fi connection uses WPA2-PSK authentication and is managed through the ESP event loop.

The event loop handles connection and disconnection events and retries up to a configurable maximum until it errors.

The firmware creates a TCP socket and connects to the fingerprinter server, but lwIP does not come with the TCP timestamps option enabled by default. TCP timestamps options defined in RFC 1323 causes every TCP packet sent by the ESP32 to include a timestamp value (TSval) in its header. On standard Linux or Windows systems, this option is enabled by default or controllable through some system setting. Initial Wireshark testing proved this to be true, verifying that the ESP32 packets do not include timestamps, while the linux server did.

The ESP-IDF menuconfig did not expose a configuration for enabling TCP Timestamps and adding it to sdkconfig.defaults would have no effect. However, the lwIP source code itself included a macro called LWIP_TCP_TIMESTAMPS which could be defined without having to edit the source code by using cmake. TCP timestamp support is enabled by setting the LWIP_TCP_TIMESTAMPS compile-time flag in the project's CMakeLists.txt:

```
idf_build_set_property(COMPILE_DEFINITIONS
    "LWIP_TCP_TIMESTAMPS=1" APPEND)
```

This flag instructs the lwIP stack to include the TCP timestamps option (RFC 1323) in all outgoing packets by appending the define to ESP-IDF's global list of compiler definitions. MINIMAL_BUILD_ON was removed from the root CmakeLists.txt to ensure the compiler did not ignore the newly added define. The TSval field is a 32-bit counter driven by the device's internal oscillator, incrementing at 1000 Hz by default in the ESP-IDF lwIP configuration.

Once connected, the firmware enters a loop in which it sends a fixed PING\n message to the server every 1000 milliseconds using vTaskDelay. This produces a steady stream of TCP packets, each carrying an updated timestamp value, at a rate of approximately one packet per second.

4.3 Fingerprinter (Measurement System)

The fingerprinter consists of multiple Python scripts that run concurrently on the measurement device.

TCP Server. The server.py script opens a TCP socket bound to port 12345 and listens for an incoming connection from the ESP32. Once connected, it enters a receive loop to keep the connection alive. Its primary purpose is to sustain the TCP session so that the ESP32 continues transmitting. The actual measurement is performed by the capture script operating at the packet level.

Scappy Env. The setup.py script has to be run in the terminal before capture.py as it sets up the necessary tools for capturing packets. It creates a python virtual environment called fingerprinter-env in the user's home directory to keep dependencies isolated from the system Python. It installs Scapy, which is a python library used for capturing and analyzing network packets, in the virtual environment. Finally, it provides instructions on how to run capture.py using run.py once the environment is ready.

Launcher Script. The run.py script acts as a argument handler and launcher for capture.py. It accepts four CLI arguments:

- `-src-ip` — the ESP32's IP address to filter traffic on (required)
- `-output` — where to save results (defaults to `timestamps.csv`)
- `-duration` — how many seconds to capture

- `-count` — max number of packets to capture
- At least one of `-duration` or `-count` must be provided, or it exits with an error

To acquire the raw socket permissions required by Scapy, run `.py` wraps the invocation of `capture.py` with `sudo` in Linux. For Windows, the command line or powershell terminal should be opened with administrator privileges.

Packet Capture. The `capture.py` script uses the Scapy library to perform passive packet capture on the network interface. It applies a Berkeley Packet filter (BPF) to select only TCP packets originating from the target ESP32's IP address. For each matching packet, it extracts the `TSval` field from the TCP timestamps option and records the local system time at which the packet arrived. These pairs — (`arrival_time`, `tsval`) — are accumulated in memory and written to a CSV file at the end of the capture session. The capture is bounded by either a time duration or a maximum packet count, specified via command-line arguments.

4.4 Clock Skew Estimation

Once the timestamp CSV file is generated, the next step is to process the data. We will assume that there are n samples in a trace, where $\vec{x}$ is the local timestamp array and $\vec{y}$ is `TSval` array. The first, relative timestamps are calculated by subtracting the first timestamp value from all timestamp values to create two lists of the local timestamp value and the `TSval` from the TCP packet capture. These values now start at 0 and end at X_n and Y_n respectively. The equation for calculating the relative timestamps is as follows:

$$\vec{X} = \vec{x} - x_0 \quad (1)$$

$$\vec{Y} = \vec{y} - y_0 \quad (2)$$

Once the relative timestamps are calculated, the slope of the timestamps are calculated using linear programming. Linear programming was used in [2] rather than using least-squares regression to account for network latency. Since packets propagating through a network will always add delay, if least-squares regression is used, the slope calculation will always be underestimated since the tick count (which is `TSval` or $\vec{y}$ in this example) is not affected by the network propagation delay; Only the local timestamp ($\vec{x}$ in this example) will be affected.

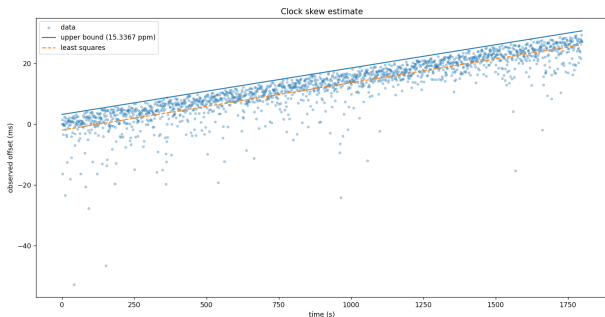


Figure 1: Clock Skew Estimation Graph

To illustrate, look at Figure 1. Using least-squares regression, the estimated slope and all measured data points are lower than the

true slope. Using linear programming, the minimum-slope upper-bounding line recovers the true slope, effectively removing the network propagation delay. This allows for a more accurate and robust calculation of the clock skew.

The result of the linear programming equation is $\alpha + \beta$ where α is the slope and β is the slope intercept that upper-bounds a set of points. Using the relative timestamp data set, the resulting equation for the linear programming constraints for all points in the set would be:

$$\alpha * x_i + \beta \geq y_i \quad (3)$$

According to [2], the linear programming solution minimizes the objective function

$$\frac{1}{|T|} * \sum_{i=1}^{|T|} (\alpha * x_i + \beta - y_i) \quad (4)$$

which minimizes the vertical distance of all the points in the trace T

Once the equation is solved, you can use the slope α to calculate the clock skew in parts-per-million (ppm) using the following equation:

$$skew[ppm] = (\alpha / TickRate[Hz] - 1) * 1,000,000 \quad (5)$$

4.5 Device Identification and Attack Detection

Once all the clock skew profiles have been established for our set of known devices, any incoming packets can be matched against these profiles by comparing their measured skews. Since clock skew is a physical property of the hardware, it cannot be altered by software reconfiguration. An attacker attempting to impersonate a device by spoofing its MAC address will still emit packets with their own device's clock skew. The skew profiling allows detecting if a device has moved location or changed networks by verifying if the IP address to which it is connected has changed. Changing IP addresses, using VPNs, connecting from different networks, or hiding behind a NAT will not change the device's skew, hence making it difficult to not be identifiable once profiled.

5 Evaluation

We conducted multiple experiments of thirty minute durations on our ESP32's.

5.1 Experimental Setup

To set up our experiments, we used our ESP32s to transmit TCP packets which are received by our host devices. The host extracted the TCP timestamp values of the packets to estimate the clock skew of the device. We ran multiple trials per device to ensure consistency.

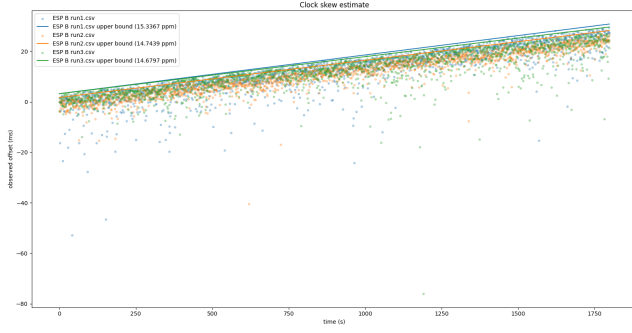


Figure 2: Clock Skew Stability

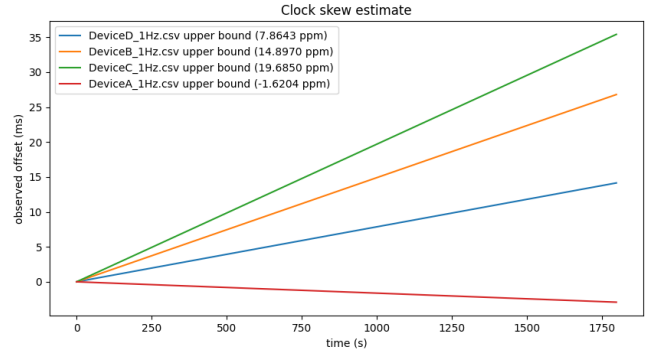


Figure 3: Clock skew estimation for one 1 Hz run per device.

5.2 Individual Test Results

Device ID	Packet Rate (Hz)	Packets Sent	Skew (ppm)
A	10	16,766	-5.64
A	10	16,671	-7.62
A	10	10,749	-5.68
B	10	14,700	3.69
B	10	14,789	3.03
B	10	14,209	3.06
C	10	13,897	5.67
C	10	13,286	6.02
C	10	14,621	6.73
A	1	1,800	-1.64
A	1	1,800	-1.62
A	1	1,800	-1.88
B	1	1,800	15.34
B	1	1,800	14.47
B	1	1,800	14.68
C	1	1,800	19.68
C	1	1,800	18.96
C	1	1,800	19.03
D	1	1,800	8.17
D	1	1,800	7.86
D	1	1,800	7.20

Table 1: 30 Minute Clock skew measurements

5.3 Combined Test Results

Figure 3 presents a clock skew estimation for four devices measured over a 30-minute observation period. The slope of each line represents the device’s clock drift relative to a reference clock.

6 Discussion

In Figure 1, you can see the results of the same ESP device across three 30-minute fingerprinting sessions. The collected offset-sets and calculated clock skews are consistent across the three runs, demonstrating the stability of the clock skew and the collection method over time. As shown in Figure 3, the clock skews of the four devices are clearly distinguishable from each other. The data points were not plotted in these graphs to improve visibility, but they follow the same trend as in Figure 1 where all the data points

lie under the minimum upper-bound. This proves that it is possible to distinguish ESP32 microcontrollers solely using their clock skew.

To evaluate stability, we ran multiple trials per device and observed that the skew values stayed within a relatively small range across runs (± 1 ppm). Longer sessions introduced more variation due to network jitter, so 30-minute runs produced the most reliable results.

One thing we noticed was repeating measurements on the same device resulted with high variability in our skew estimates. This indicated that network jitter and noise was interfering with the clock skew signal. We also observed that the total packets transmitted were closer to 15k rather than 18k (100% transmission at 10Hz for 30 minutes). Due to this, we decided to also test at a lower frequency to reduce the network traffic. This came at the cost of reducing the samples available for the regression skew estimation.

We also noticed a difference in clock skew for all four devices when running 1 Hz and 10 Hz, as seen in Table 1. Some devices had a 4 ppm difference while others had a difference of greater than 10 ppm. We are not sure if this is related to our network and methodology of data collection or if this is normal due to a vast reduction in packet loss when swapping to 1 Hz for our packet frequency.

7 Potential Impacts

The primary potential impact of this work is improved security for IoT systems. Our system provides resource-constrained platforms a device identification mechanism that requires no cryptographic infrastructure and requires no computational overhead on the monitored devices. An attacker attempting a cyber attack would need to replicate the target device’s quartz crystal oscillator characteristics which is highly unlikely to happen.

More broadly, the technique could be applied in network forensics or tracking devices across network address changes. The passive nature of the approach is particularly valuable: the fingerprintee requires no modification and does not need to be aware that it is being measured.

There are also privacy issues to consider since the same technique that can identify legitimate devices can be used to track individuals across networks. A device’s clock skew persists across reboots and location changes and can be used in malicious ways.

Any deployment of this technology should follow proper usage ethics.

8 Change of Plan

The original proposal described two phases: first, a laptop-based fingerprinting system, and second, porting the detection logic to an ESP32 to serve as an embedded monitor. Due to time constraints, the second phase has been deferred; the current implementation focuses on the laptop-based fingerprinter and validates the core measurement technique.

9 Embedded Systems Aspects

The project relied on using an ESP32, a microcontroller with built-in Wi-Fi capabilities, which makes it a common platform for IoT and embedded networking applications. The ESP32 also runs using FreeRTOS, which is a common real time operating system in embedded applications. To write firmware to the ESP32 requires setting up the ESP-IDF (Espressif IoT Development Framework), which is the official development environment for ESP32 projects. ESP-IDF provides a build system based on CMake, a menuconfig system for configuring project settings at build time, and the idf.py tool for building, flashing, and monitoring the board. These are all standard parts of an embedded development workflow.

Project-level configuration in ESP-IDF is established using `Kconfig.projbuild`, which defines menuconfig entries that get compiled into the firmware as preprocessor constants. In our project this was used to configure the Wi-Fi SSID and password, the server IP address, and the server port without hardcoding them into the source code. Enabling TCP timestamps required adding a global compile definition directly in `CMakeLists.txt`:

```
idf_build_set_property(COMPILE_DEFINITIONS
    "LWIP_TCP_TIMESTAMPS=1" APPEND)
```

This modifies the build-time configuration of the lwIP network stack, which is the TCP/IP implementation used by the ESP32. Editing CMake files to set global defines that affect library behavior is a characteristic of embedded C development since as developers, we have direct control over the toolchain and build system.

On the firmware side, the ESP32's Wi-Fi driver is managed through an event loop provided by ESP-IDF. The firmware does not poll for connection status, it registers event handlers that trigger when the Wi-Fi connects, disconnects, or obtains an IP address. This event-driven model is common in embedded systems because blocking the main task while it waits for hardware events is undesirable.

Once connected to Wi-Fi, the firmware creates a TCP socket using the lwIP socket API and connects to the measurement server. This is the same BSD socket interface used on desktop systems, but running on an embedded network stack on a microcontroller. On the desktop side, `server.py` creates a TCP socket, binds it to a port, and listens for incoming connections from the ESP32, completing the embedded-to-desktop communication channel.

The fingerprinter operates at the network packet level using Scapy. It captures raw TCP packets and extracts fields from the TCP options layer. Working directly with network layer packets and parsing data from them can be relevant to IoT and embedded networking.

All code and data can be accessed using the GitHub link in [3].

10 Task Breakdown

Husam Chekfa was responsible for the ESP32 firmware (`fingerprinter.c`), including Wi-Fi connection management, TCP socket setup, lwIP timestamp configuration via the `LWIP_TCP_TIMESTAMPS` compile flag, and periodic packet transmission.

Michael Antabian was responsible for the fingerprinter system, including the TCP server (`server.py`), the Scapy-based packet capture script (`capture.py`), the virtual environment setup script (`setup.py`), and the `run.py` wrapper.

Dalen Ricks (Team Lead) was responsible for the clock skew estimation and analysis, including the linear programming-based skew calculation, graphing of offset-set data, and comparison of skew values across devices.

All three team members contributed to the experimental setup, conducted the packet capture experiments and collected the timestamp data. All three team members contributed to this report.

11 Relation to Other Project

Not applicable.

References

- [1] Ding-Jie Huang, Wei-Chung Teng, Chih-Yuan Wang, Hsuan-Yu Huang, and Joseph M. Hellerstein. 2008. Clock Skew Based Node Identification in Wireless Sensor Networks. In *IEEE GLOBECOM 2008 - 2008 IEEE Global Telecommunications Conference*, 1–5. <https://doi.org/10.1109/GLOCOM.2008.ECP.363>
- [2] Tadayoshi Kohno, Andre Broido, and K.C. Claffy. 2005. Remote Physical Device Fingerprinting. In *2005 IEEE Symposium on Security and Privacy (SP'05)*, 211–225. <https://doi.org/10.1109/SP.2005.18>
- [3] Antabian, Chekfa, Ricks, "GitHub - DalenRicks/ECE-568-Project," GitHub, 2026. <https://github.com/DalenRicks/ECE-568-Project> (accessed May 08, 2026)